SACT: Automatic Cluster-Tree Construction for Very Large Spatial Databases*

Aoying Zhou§, Weining Qian§†, Hailei Qian§, Zhiyuan Cai§, Wen Jin‡
{ayzhou, wnqian, hlqian, zycai}fudan.edu.cn, wjin@cs.sfu.ca

§Department of Computer Science and Engineering, Fudan University
200433, Shanghai, P.R. China
‡Simon Fraser University, 8888 University Drive, Burnaby, B.C., V5A 1S6, Canada

Abstract

The aim of clustering methods is to partitioning data sets into clusters so that intra-cluster data are similar and inter-cluster data are dissimilar. To a large extent, these methods focus more on performance than on the issues of usability and explanatory, which are essentially quite important in real applications. As a result, the task of data analysis and data mining will suffer some problems from the fact that: 1) Most available algorithms need users to input a set of complex parameters; 2) The result of clustering is usually represented as a bunch of *flat* clusters containing related data. Furthermore, the challenge in efficiently clustering very large databases does still exist. To attack above problems, we propose in this paper a novel approach for spatial data clustering, which has the advantages that: 1) Based on MDL(minimum description length)-principle, the nesting relationship of clusters are naturally represented as a **cluster-tree**, so that the clustering results are easily understanding; 2) This method needs no parameter and is taken automatically by meta-learning, which describes the clustering behavior precisely. 3) The dynamic merging operation in clustering, which actually compresses data in summarization, can achieve a good scalability in handling very large databases. The analysis and experiments show that our algorithm can work in very large spatial databases efficiently and effectively.

1 Introduction

Clustering is the process to partition a data set into groups (called clusters), so that intra-cluster data are similar and inter-cluster data are dissimilar. Clustering is an important data mining technique which is widely applied in spatial databases [6]. Clustering is usually viewed as an unsupervised learning process, because that users know little about the characteristics of data sets a priori in most cases.

Although current clustering algorithms needn't training data, they are difficult for using and understanding. This problem lies in two aspects. Firstly, most algorithms are sensitive to parameters, while appropriate parameter setting can only be acquired when the property of data set is known. Since it is an unsupervised process, inappropriate parameter setting always leads to unsatisfactory clustering result. Secondly, the result of clustering is usually a bunch of clusters, within which are a group of data objects. In other words, the result is a set of *flat* clusters, which is difficult for understanding. Since different applications may need different cluster granularity while one application may want to analyze the subcluster of a cluster, discovering the cluster-structure is a more general problem than traditional cluster analysis.

As intelligible, scalability is another basic request to all clustering algorithms. Although some effort has been made to target at it, it is not well resolved. Some methods sacrifice resolution of clusters [19, 23, 25], while some others give up the ability to identify small clusters [6].

*This work is partially supported by the National Key Fundamental Research Program (G1998030414) and NSFC (60003016).
†The author is supported by Microsoft Research Fellowship.

1.1 Our Contribution

In this paper, an algorithm, SACT (*Scalable Automatic Cluster Tree construction*), which is designed to cluster very large spatial databases automatically while preserving the effective and efficiency, is presented. In other word, it is desgined to obtain *high accuracy*, *intelligible*, *automation*, and *scalability* simultaneously. It has the advantage over other clustering analysis algorithms in that: 1) A cluster-tree, other than flat clusters, is generated, so that the cluster-structure is captured accurately; 2) The representation of clusters is generated dynamically according to the complexity of the clusters; 3) It can scale up to very large spatial databases smoothly while preserving the effectiveness and efficiency. The first two advantages make the result understandable, while the last one makes the algorithm applicable to real applications.

First of all, a cluster-tree is a tree, while each node in it denotes a cluster, and the parent-child relationships denote the nesting relationships between clusters. Since cluster-tree is designed to describe the cluster-structure, it should satisfy the following conditions:

Distinction: Each data point in a cluster is distinct from those points in its siblings while compared with the points within that cluster.

Completeness: All data groups satisfy the distinction condition have corresponding nodes in the cluster-tree.

We define a measurement, *cluster-factor*, to evaluate the distinction of clusters, which can be calculated efficiently when using hierarchical framework. Meanwhile, the completeness is guaranteed by using MDL (Minimal Description Length)-based method.

The algorithm is designed to process the clustering task automatically, which means it needn't user to set parameters. The needed parameters, namely parameters for cluster-reprensentation and stop-condition, is learned based on analysis of the data set. The analysis and experiments show that this *meta*-learning process only cost little overhead comparing with traditional hierarchical clustering algorithms.

Furthermore, the algorithm takes an partitioning $\rightarrow$ summarization $\rightarrow$ result-merging framework, so that the data set only need to be load from disk to memory for once. And the final result-merging process, which is also automated by taking advantage of the summarization information (e.g. cluster-trees), is almost none-cost in the whole process. Thus, the algorithm acquires high-scalability while preserving the high-accuracy and automation.

1.2 Paper Organization

The rest of this paper is organized as follows: In section 2, the algorithm framework is introduced. MACT (memory cluster-tree construction), which is the key step to generate intelligible cluster-tree is introduced and analyzed in detail in section 3, while in section 4 we extend MACT to SACT, so that the algorithm can process very large databases efficiently. Theoretical analysis and empirical study of the algorithm are given in section 5. After the introduction to related work in section 6, finally, section 7 is for concluding remarks.

2 Algorithm Framework

The overall framework of SACT is illustrated in Figure 1. The first phase partitions the data set so that each partition can be processed in memoy efficiently. Therefore, each data object only need to be load into memory once. After a partition is loaded into memory, the MACT algorithm, which is designed to automatically cluster data in memory, construct a cluster-tree for the partition. The last phase merges all the cluster-trees built for the partitions, so that the final cluster-tree is obtained. The parameters for merging the cluster-trees are determined based on the statistics provided by each cluster-tree. Therefore, as the MACT algorithm, the tree-merging phase is also automatic. Consequently, the whole process depends little on user-provided parameters.

The most advantage of the framework over other algorithms is that for each partition, a cluster-tree, other than flat cluster, is constructed. Thus, it provides flexibility for further adjusts in the final tree-merging phase. As the result, it can build accurate cluster-tree automatically while attaining high scalability.

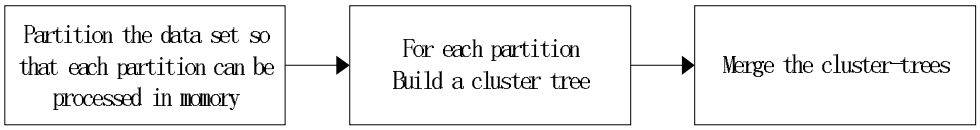


Figure 1: The framework of SACT

Table 1: The process record

inner_dist	$\frac{u.inner_dist \times (u.size-1)+v.inner_dist \times (v.size-1)+distance(u,v)}{u.size+v.size-1}$
merge_dist	$\min\{distacne(w,x) \mid x \text{ is a } sub-cluster \text{ other than } u \text{ and } v\}$
rep_distrib	$\max\{\min_2\{distance(o,p) \mid p \text{ is a representative of } w, \text{ and } o \neq p\} - \min_1\{distance(o,q) \mid q \text{ is a representative of } w, \text{ and } o \neq q\} \mid o \text{ is a representative of } w\}\}$
rep_density	$\frac{1}{\min\{distance(p,q) \mid p \text{ and } q \text{ are representatives of } w, \text{ and } p \neq o\}}$
diameter	$\max\{distance(p,q) \mid p \text{ and } q \text{ are representatives of } w, \text{ and } p \neq q\}$
size	$w.size = u.size + v.size$

The purpose of partitioning is to segment a data set into groups so that each group can be proessed in memory. There is an important request for partitioning: each final cluster (or part of it) should be distinct in at least one partition. In other words, each cluster or one of its subcluster should be found in the cluster-tree according to certain group. Furtunately, the following lemma holds:

Lemma 1 *For any area A, B in original data space, if $density(A) < density(B)$, then for the partitioned groups, there exist at least one group i holds that: $density_i(A) > density_i(B)$.*

Therefore, a cluster (or a sub-cluster of it) will be at least distinct in one of the groups.

In the following two sections, we will introduce the second and third phase of our algorithm detailly in order.

3 Automatic Cluster-Tree Construction in Memory

The algorithm for building cluster-tree for each partition is called MACT, which means *Memory Automatic Cluster Tree construction*. As it is named, MACT can build the cluster-trees without the parameters for cluster representation and stop-condition. This is the major difference between MACT with other hierarchical clustering algorithms, which usually need user to specify the stop condition, while sometime generate either insufficient or redundant cluster representation.

3.1 Process Record

Process record is used to record information about merging operations for further analyzing. It is used to determine both the cluster representation and stop condition, as will be presented in the next two subsections.

To simplify the following discussion, some basic concepts about hierarchical clusteing is introduced first. A sub-cluster is a set of data objects. A merging operation merges two sub-cluster u and v into a new sub-cluster can be denoted simply by w, which is the sub-cluster it generates. The process of hierarchical clustering is a sequence of merging operations, which merges the initial stand-alone data objects into final clusters. For each merging operation, we generate a tuple for process record, which contains the information in Table 1.

We explain the items in process record one-by-one as follows (it is assumed that the record is for a sub-cluster w):

inner_dist denotes the average inner distances between objects within w.

merge_dist denotes the probable distance between a w and the sub-cluster it will be merged with.

rep_ distrib denotes the degree of uniformity of representatives in w, as it will be explained later.[1]

rep_ density denotes the density of representatives in w.

diameter denotes the largest distance of pair of objects in w, which can be treated as a kind of measurement for the scale of w.

size denotes the number of objects in w, which is another kind of measurement for the scale of w.

Furthermore, if a sub-cluster w is considered as a final cluster, its *inner_dist* is defined equal to its *merge_dist*. After the process record is defined, *cluster factor*, which denotes the degree of a sub-cluster being a cluster, can be defined as follows:

Definition 1 *For a sub-cluster w, its cluster factor is: w.cluster_factor=w.merge_ dist/w.inner_ dist.*

Intuitively, sub-clustrs with small $cluster_factor$ are not clustrs in any granualrity, since they are merged with some data (sub-clusters) close to them.

Process record is recorded and cluster factors are calculated to determine the representation and final granularities of clusters.

3.2 Cluster Representation Generation

Multi-representative technique has the ability to identify arbitrary-shaped clusters [6]. However, for complex-shaped clusters or clusters close to other data objects, large amount of representatives are always need. However, users cannot know how many representatives are sufficient in a priori. Moreover, wrong setting of number of representatives may lead to unsatisfiable result: setting it too small will produce wrong clustering results for complex-shaped clusters, while setting it too large will greatly increase the running time, since the algorithm should search a k-d-tree, which contains all the representatives, frequently [6].

Different with CURE, two heuristics are used to judge when the representatives are enough for a cluster:

Heuristic for distribution of representatives if for a cluster, *rep_ distrib* is larger than $\frac{1}{rep_density}$, the representatives are not enough, since a representatives are not uniformly distributed in the clusters, so that they may be not able to describe the shape of the cluster precisely.

Heuristic for density of representatives if for a cluster, *merge_ dist* is less than $\frac{1}{rep_density}$, the reprentatives are not enough, since they are not dense enough to distinguish data within the cluster from data out of it.

Therefore, only when those two conditions are not satisfied, the representatives for a cluster are not added anymore.

3.3 Cluster Determination

MDL(Minimum Description Length)-based method is employed to determine the clusters that should appear in the final cluster-tree. The basic idea behind MDL principle is that: the best model to describe data is the model with minimum description length, in which description includes the coding for both data and model[16]. Intuitively, it is an instance of the idea *simple is the best.*

Here, the model for a clustering process is the cluster-tree, while the data are the merging operations and the cluster factors, which will be used for further analysis. The description length is calculated as follows:

$$\begin{aligned} DL_{model} &= DL_{cluster} + DL_{link} \\ &= DL_{atomic_cluster} + DL_{nonatomic_cluster} + DL_{link} \\ &= DL(representatives) + DL(pointers_to_representatives) + DL(link) \end{aligned}$$

$$DL_{data} = DL(cluster_factors)$$

[1] min_1 gets the minimum value in the set, while min_2 gets the second minimum value in the set

$$DL_{overall} = DL_{model} + DL_{data}$$

Since the limit of space and the calculation of description for every part is straightforward and trivial, the formulas are not expanded further.

However, since there is 2^n possibilities for merging operations to be clusters, it is time consuming to try all the possibilities. Therefore, an approximate approach is choosed, as it is listed below:

```
1  procedure find_clusters ()
2     for i in process record in the order they appear
3        if DL(i as a cluster) < DL(i as a non-cluster)
4           mark i as cluster;
5        else
6           mark i as non-cluster;
```

This approach considers a sub-cluster each time to judge whether it is a final cluster. Since the sub-clusters are considered in the order they appear, only the ones have already determined will be used to judge current sub-cluster that is considering. Although it is an approximate method, it is effective, as will be shown in the experiments.

The cluster-tree building based on identified clusters is straightforward. Since the limit of space, we omit the details here.

4 Cluster-Tree Merging

The MACT algorithm introduced in the last section assumes that the data set can be processed in memory, which is not true in most cases. As it is introduced in framework, cluster-tree merging phase extends MACT to SACT, so that very large databases can be processed efficiently.

To generate the final cluster-tree, cluster-trees corresponding to partitions are merged one-by-one. A level-wise, bottom-up method is employed to merge a pair of cluster-trees. Firstly, the nodes in a cluster-tree is sorted in hierarchical order.

Definition 2 *The hierarchical order of a tree is the order of nodes that preserve the ascendant order of the level of nodes, in which the level of a node a in a tree $l(a)$ is defined as follows:*

$$l(a) = \begin{cases} 1 & a \text{ is a leaf} \\ \max\{l(b) | b \text{ is a child of } a\} + 1 & a \text{ is not a leaf} \end{cases}$$

Lemma 2 *If node a is a child of node b, then $s(a) < s(b)$, in which $s(i)$ denotes the order of node i in the hierarchical order.*

Definition 3 *Two nodes a_1 and a_2, in which a_1 is in cluster-tree T_1 and a_2 is in cluster-tree T_2, $l(a_1) = l(a_2)$, are connected, if: $distance(a_1, a_2) < min\{a_1.merge_dist, a_2.merge_dist\}/\min\{CFT_i\}$, in which:*

$$\begin{aligned} CFT_i \quad = \quad & (\min\{cluster_facor_j | j \text{ is a cluster in partition } i\} - \\ & \max\{cluster_facor_j | j \text{ is a non-cluster sub-cluster in partition } i\})/2 \end{aligned}$$

Two connected nodes denote that they are so close that they can be treated as in the same cluster in corresponding level. Therefore, two connected nodes should be merged. Furthermore, since the the merging condition is defined based on all the cluster-trees, the final cluster-tree will satisfy the lowest condition of distinction. Thus, following method is applied to merge a pair of cluster-trees.

```
1  procedure ClusterTreeMerging (T1, T2)
2     T:=Φ;
3     for i:=1; i<max{level(T1),level(T2)}; i++
4        for each pair of nodes (T1.a,T2.b) satisfying that l(a)==l(b)==i
5           if a and b are connected and
              each child of a is merged with at least one child of b and vice versa
6              mark a and b as should-be-merged;
7        for each set of should-be-merged nodes A={a1, a2, ..., ak}
```

Table 2: Time complexity of SACT

	Time complexity	Comments
Hierarchical clustering	$O(m^2 \times \log m)$	m is the number of data objects that can be processed in memory, processed once for every partition
Cluster-tree determination	$O(k_{\max} \times m)$	$k_{\max}$ is the maximum number of atomic clusters among all tree, processed once for every partition
Cluster-tree merging	$O(k_{\max}^2)$	The tree merging process is executed $\lceil n/m \rceil - 1$ times

```
8           w:=merge(A);
9           T.insert(w);
10    return T;
```

It should be noted that the tree structure always maintain a root if it is non-empty. The implementation details are omitted here.

5 Analysis and Experimental Results

5.1 Analysis

The time complexity of each phase is shown in Table 2. Therefore, the time complexity is: $O((n/m) \times m^2 \times \log m) + O((n/m) \times k_{\max} \times m) + O((n/m) \times k_{\max}^2)$. Since for very large spatial databases, $k_{\max}$ will not be large when compared with n, it can be treated as a constant. Furthermore, if the ability of memory storage is fixed, m is a constant. Therefore, the time complexity is $O(n)$, which is linear to the size of the data sets to be processed. Furthermore, the data set is read from disk almost only once, since the size of cluster-trees is much smaller than the size of data set. So the algorithm can be smoothly scaled to very large databases.

The accuracy of the cluster-tree built by SACT is garanteed by following reasons. 1) The multi-representative technique can identify most simple-shaped clusters. 2) The representation tuning technique makes sure that representatives are sufficient to distinguish a cluster from other data. Therefore, even complex-shaped clusters can be identified. 3) The MDL-based automatic cluster identification process accurately finds the clusters with distinct large cluster-factor, which means that they are distinct to other data. 4) The cluster-tree merging process makes sure that the requests for an accurate cluster-tree are still satisfied by the final cluster-tree.

Furthermore, the final cluster-tree is explicable. The parent-child relationships in the cluster-tree denote the nesting relationships of clusters, which provide the information about granularities of clusters. Clusters with small size are the outliers, which may be interested for further study in some applications. Clusters with large amount of representatives are either large or complex-shaped or both.

However, SACT is sensitive to noises, since they will greatly reduce *merge_dist* of each cluster. However, by using density-based noises elimination technique, such as the method introduced in [3], the noises can be removed easily.

5.2 Experimental Setup

The experiment environment is two PIII 755MHz PC workstations each with 128MB RAM running Microsoft Windows 2000 Professional. Some of the test data sets are simulation of the test data sets in [6, 24]. Others are generated by hands. Since our purpose is to test the ability of the algorithm to find well-separated clusters automatically while being high scalable, the density-based noises elimination[3] is applied as preprocessing.

The purposes of the experiments are as follows:

- Validating the effectiveness of the automatic cluster-tree building ability;
- Validating the effectiveness of the cluster representation tuning technique;
- Comparing the performance and scalability of our algorithm with those of CURE.

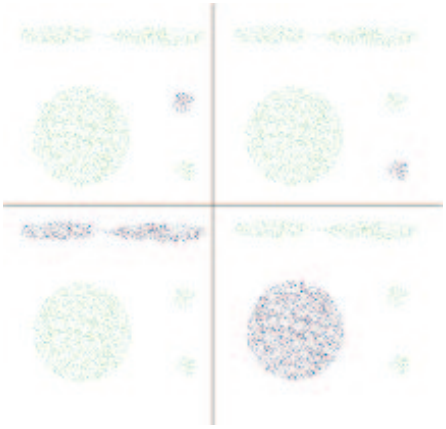


Figure 2: Clustering Result of data set with dumbbell-shaped cluster

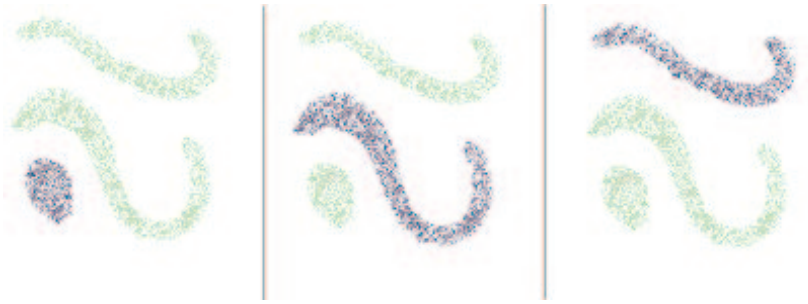


Figure 3: Clustering result of data set with curse-shaped clusters

The experimental results are provided in the following two sub-sections.

5.3 Clustering Effectiveness

We constructed several two-dimensional data sets, within which clusters are well-seperated, as it is shown from Figure 2 to 9. In Figure 2 to 7, each figure shows the result of a data set. Each part of the figure shows a cluster, whose data points are denoted by darker color than other data points. This method is chosen to show the result because that some data points may appear in different clusters in the cluster-tree. Figure 8 and 9 use different methods, since there are too many clusters to be identified. In Figure 8, each black area denotes an atomic cluster found by SACT. The clusters in the same circle form a high-level cluster. Therefore, it denotes a complex cluster-hierarchy. In Figure 9, each cluster is denoted by one color. There is no high-level clusters found.

Clusters with arbitrary shapes cannot be identified by traditional algorithms such as k-means or BIRCH, while they can only be identified by CURE when parameters are appropriately set. We test three data sets containing complex-shaped clusters, namely, dumbbell-shaped, curve-shaped, and hollow-shaped clusters. The result is shown in Figure 2, 3, and 4. They show that these complex-shaped clusters are correctly identified. It should be noted that no parameter-setting are needed in advance. The number of representatives for the clusters are shown in Table 3. This experimental result supports that our cluster representation generation strategy can generate sufficient while concise representation for clusters, so that automatic clustring is well-supported.

Another four data sets with hierarchical clusters are tested, as they are shown in Figure 5 to 8. The result shows that the high-level clusters are accurately identified as well as those atomic clusters. Furthermore, a data set with many atomic clusters but containing no high-level cluster is tested, as it is shown in Figure 9. The result shows that no high-level cluster is identified. Figure 8 and 9 show that SACT can build accurate cluster-tree for data set whether it contains complex cluster hierarchy or contains no cluster hierarchy.

Furthermore, we test several complex data sets with noises. The U-statistics, which is usually used

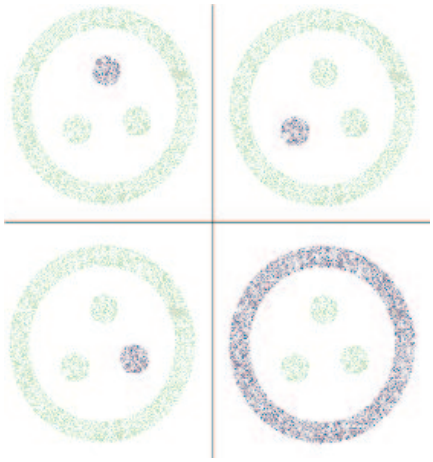


Figure 4: Clustering result of data set with hollow-shaped cluster

Table 3: Numbers of representatives for clusters

Data set	Cluster description	Number of representatives
Figure 2	Dumbbell-shaped	33
	Large ball	57
	Small ball I	8
	Small ball II	9
Figure 3	Curse-shaped I (close to the ball)	84
	Curse-shaped II	50
	Ball	10
Figure 4	Hollow-shaped	123
	Ball I	9
	Ball II	12
	Ball III	11

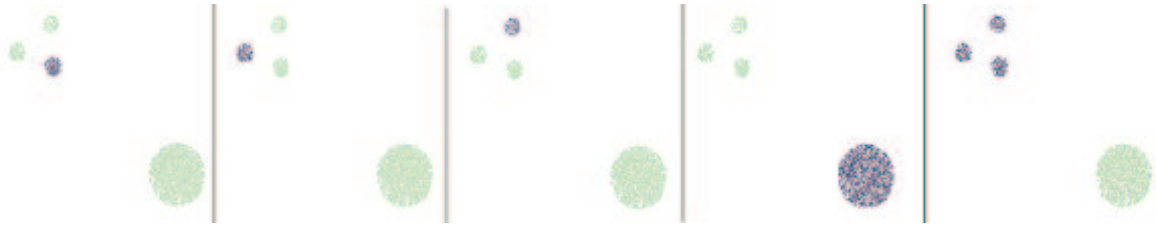


Figure 5: Clustering result of data set with simple cluster-hierarchy

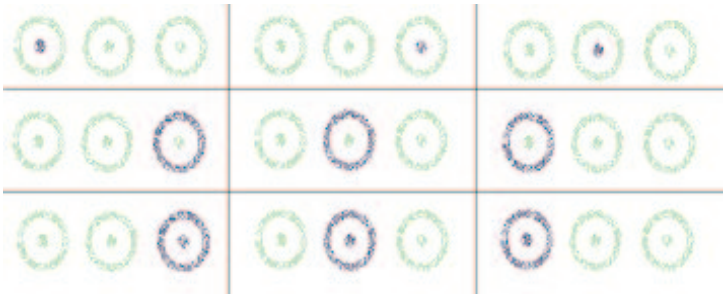


Figure 6: Clustering result of data set with balanced cluster-hierarchy

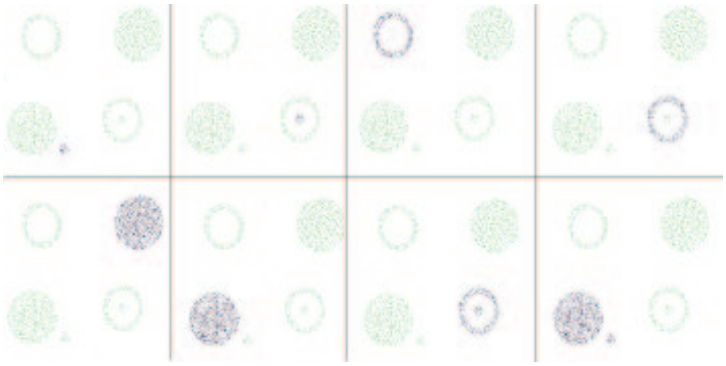


Figure 7: Clustering result of data set with non-balanced cluster-hierarchy



Figure 8: Clustering result of data set with complex cluster-hierarchy

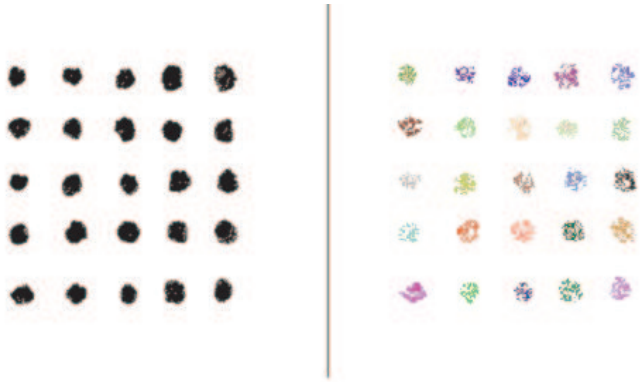


Figure 9: Clustering result of data set with no cluster hierarchy

Table 4: Average U-statistic comparison of CURE and SACT

	Average U-statistic (CURE)	Average U-statistic (SACT)
Data set 1	61,784,184	61,136,992
Data set 2	427,516,462	347
Data set 3	230	5

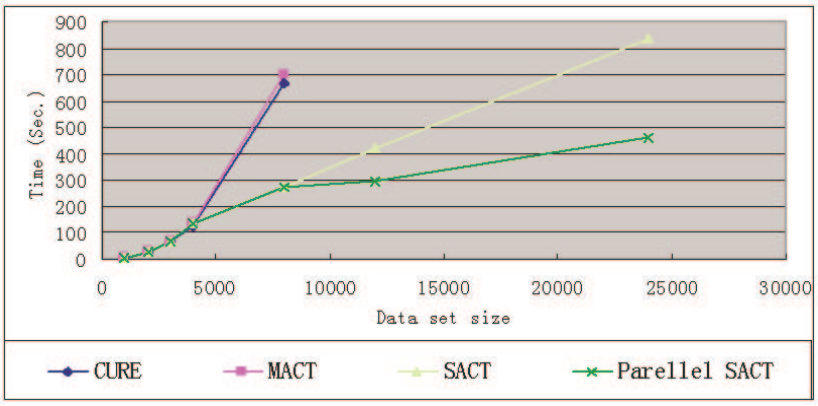


Figure 10: Performance comparison of CURE, MACT, and SACT

to evaluate the degree of dissimilarity of clusters to other data, is used to quantitatively compare SACT with CURE. If a cluster is well-seperated from other data or clusters, the statistic U should be small [12]. We use the average of U-statistic of clusters found by the algorithms to test the quality of the algorithms. If the value is small, it means that the identified clusters are well-seperated from other data. In other words, the smaller the average U-statistic value is, the more distinct the clusters are. The result is shown in Table 4. For SACT, only atomic clusters are used to compute U-statistics, since CURE can only build flat clusters. For CURE, we set the parameter like this: the number of final clusters is equal to the number of atomic clusters found by SACT, the number of representatives for each clusters is set obey the rule introduced in [22]. The result shows that SACT is better than CURE. This result is important since it prove that automatic clustering can gain even higher quality result than traditional algorithms do.

5.4 Scalability

The performance of MACT, SACT, and CURE[6] is compared. For CURE, the number of representatives in each cluster is set to 4 (the suggested number in [22]), while k is set to the number of atomic clusters found automatically by SACT.

The buffer is set to the size to be able to process 4000 data points. The experimental result which compares the performance of CURE, MACT, SACT and parallel SACT is shown in Figure 10. It is shown that when the data can be processed in memory, CURE is a little faster than MACT, since MACT need additional overhead for determining the cluster representation and building the cluster-tree. When the data cannot be processed in memory, the advantage of SACT over CURE or MACT is distinct. The parallel SACT applies MACT on partitions parallelly on two PCs, and merges the cluster-tree finally on one computer. The result shows that it can improve the performance further. SACT and parallel SACT are much faster than CURE and MACT since that the data sets need to be load only once.

6 Related Work

Clustering is widely studied in data mining community [1, 2, 4, 6, 7, 8, 9, 10, 11, 13, 14, 17, 18, 19, 21, 23, 24, 25]. Among the research work, hierarchical clustering is a traditional approach. Although current hierarchical clustering algorithms can deal with complex clusters well, they suffer the performance problem, especially when processing very large databases, since some kernal data structures are

designed for memory-resident data, such as the heap and k-d-tree employed in CURE, ROCK, and CHAMELEON [6, 7, 13]. Some effort has been made to improve the scalability of the algorithms, which can be classified into three categories: sampling [6, 13], partitioning [6, 13, 23], and summarization [23, 25]. However, since detailed cluster information are igored in sampling and summarization methods, small clusters may be lost, while subsequent analysis is impossible. Partitioning methods are widely employed in cluster analysis. Existing algorithms taking this approach sometimes can cluster accurately within each partition. Nevertheless, the final result should be globally analyzed. Precise sub-clusters in partitions may not deduce precise clusters satisfying overall requests.

Different with these methods, the summarization information generated by MACT is muli-level hierarchies, which can be studied in different granularities. Therefore, it is more flexible than traditional summarization techniques. Meanwhile, the cluster-tree merging process utilizes the characteristics of all cluster-trees. Therefore, it overcomes the locality shortcoming of traditional partitioning methods.

In recent years, some algorithms targeted at building meaningful cluster structures have been developed, such as ISAAC [21], H-BLOB [20], and OPTICS [2]. Different with SACT, ISAAC needs user to input a series of values which lies in [0, 1]. Since the domain of the parameter is a continueous range, user cannot set appropriate values easily, which greately effect the quality of the clustering result. H-BLOB is more like a visualization technique than data mining algorithm. OPTICS, which builds cluster structure based on density information, can cluster the data set semi-automatically, with the interaction of users. However, it is density-based, while SACT is distance-based.

Detailed surveys about clustering techniques are provided in [5, 14, 26].

7 Conclusions and Future Work

The theoretical analysis and empirical study show that SACT can automatically build accurate cluster-trees for very large databases efficiently. It has the advantage over other clustering methods that: 1) It can build meaningful cluster-trees, which provide the information about cluster-structure that is useful for further study. 2) MACT algorithm learns the parameters from the data set to make the process automatic, which means that no parameters are needed. 3) The process only read the data set from disk once, so that it can be scaled to very large databases smoothly. 4) The automatic cluster-tree building method can gain even better result than traditional clustering algorithms, while being efficient. Therefore, SACT is appropriate for mining very large spatial databases.

Currently, SACT still use noises elimination as stand-alone preprocessing. To integrate the noise-handling into the algorithm is a straightforward future work. Meanwhile, extending current cluster-structure mining method to processing high-dimensional data is another novel topic.

References

[1] R. Agrawal, j. Gehrke, D. Gunopulos, and P. Raghavan. Automatic Subspace Clustering of High Dimensional Data for Data Mining Applications. In *Proc. of ACM SIGMOD'98 Conf.*, 1998.

[2] M. Ankerst, M. M. Breunig, H-P. Kriegel, and J. Sander. OPTICS: Odering Points To Identify the Clustering Structure. In *Proc. of ACM SIGMOD'99 Conf.*, pp.49-60, 1999.

[3] M. M. Breunig, H-P Kriegel, R. T. Ng, J. Sander: LOF: Identifying Density-Based Local Outliers. In *Proc. of ACM SIGMOD'2000 Conf.*, pp.93-104, 2000.

[4] M. Ester, H-P. Kriegel, J. Sander, and X. Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noises. In *Proc. of 2nd Int'l Conf. on Knowledge Discovery and Data Mining*, 1996.

[5] D. Fasulo. An Analysis of Recent Work on Clustering Algorithms. Available at: http://www.cs.washington.edu/homes/dfasulo/clustering.ps , 1999.

[6] S. Guha, R. Rastogi, and K. Shim. CURE: An Efficient Clustering Algorithm for Large Databases. In *Proc. of ACM SIGMOD'98 Conf.*, 1998.

[7] S. Guha, R. Rastogi, and K. Shim. ROCK: A Robust Clustering Algorithm for Categorical Attributes. In *Proc. of the 15th Int'l Conf. on Data Engineering (ICDE'99)*, 1999.

[8] E. H. Han, G. Karypis, V. Kumar, and B. Mobasher. Clustering in a High-Dimensional Space Using Hypergraph Models. *Bulletin of the Technical Committee on Data Engineering*, 21(1), 1998.

[9] A. Hinneburg, and D. A. Keim. An Efficient Approach to Clustering in Large Multimedia Databases with Noise. In *Proc. of the 4th Int'l Conf. on Knowledge Discovery and Data Mining*, 1998.

[10] A. Hinnebrug, and D. A. Keim. Optimal Grid-Clustering: Towards Breaking the Curse of Dimensionality in High-Dimensional Clustering. In *Proc. of the 25th Int'l Conf. on VLDB*, 1999.

[11] Z. Huang. Extensions to the K-means Algorithm for Clustering Large Data Sets with Categorical Values. *Data Mining and Knowledge Discovery*, 2: pp.283-304, 1998.

[12] Z. Huang, M. K. Ng, D. W.-L. Cheung. An Empirical Study on the Visual Cluster Validation Method with Fastmap. In *Proc. of DASFAA 2001*: pp.84-91, 2001.

[13] G. Karypis, E. H. Han, and V. Kumar. CHAMELEON: A Hierarchical Clustering Algorithm Using Dynamic Modeling. To appear in *IEEE Computer: Special Issue on Data Analysis and Mining*, 1999.

[14] L. Kaufman, and P. J. Rousseeuw. *Finding Groups in Data: An Introduction to Cluster Analysis.* John Wiley & Sons, 1990.

[15] D. A. Keim, and A. Himmeburg. Clustering Techniques for Large Data Sets - From the Past to the Future. *KDD Tutorial Notes*, 1999.

[16] M. Mehta, J. Rissanen, and R. Agrawal. MDL-based Decision Tree Pruning. In *Proc. of 5th Int'l Conf. on KDD*, 1995.

[17] R.T. Ng, and J. Han. Efficient and Effective Clustering Methods for Spatial Data Mining. In *Proc. of the 20th Int'l Conf. on VLDB*, pp.144-155, 1994.

[18] J. Sander, M. Ester, H-P. Kriegel, and X. Xu. Density-Based Clustering in Spatial Databases: The Algorithm GDBSCAN and its Applications. *Data Mining and Knowledge Discovery*, Kluwer Acedemic Publicashers, Vol. 2, 1998.

[19] G. Sheikholeslami, S. Chatterjee, and A. Zhang. WaveCluster: A Multi-Resolution Clustering Approach for Very Large Spatial Databases. In *Proc. of the 24th Int'l Conf. on VLDB*, p.428-438, 1998.

[20] T.C. Sprenger, R. Brunella, M. H. Gross: H-BLOB: A Hierarchical Visual Clustering Method Using Implicit Surfaces. Technical Report No. 341, Computer Science Department, ETH Zürich, 2000. Available at: ftp://ftp.inf.ethz.ch/pub/publications/tech-reports/3xx/341.pdf

[21] L. Talavera, and J. Bejar. Efficient Construction of Comprehensible Hierarchical Clustering. In *Proc. of PKDD'98*, pp.93-101, 1998.

[22] http://cs.kaist.ac.kr/~shim/sigmod98.ps

[23] W. Wang, J. Yang, and R. Muntz. STING: A Statistical Information Grid Approach to Spatial Data Mining. In *Proc. of the 23rd Int'l Conf. on VLDB*, p.186-195, 1997.

[24] X. Xu, M. Ester, H-P. Kriegel, and J. Sander. A Distribution-Based Clustering Algorithm for Mining in Large Spatial Databases. In *Proc. of the 14th Int'l Conf. on Data Engineering (ICDE'98)*, pp.324-331, 1998.

[25] T. Zhang, R. Ramakrishnan, and M. Livny. BIRCH: An Efficient Data Clustering Method for Very Large Databases. In *Proc. of ACM SIGMOD'96 Conf.*, 1996.

[26] A. Zhou, W. Qian, H. Qian, and W. Jin. Observing Clustering Techniques from Different Viewpoints. Technical Report, Computer Science Department, Fudan University, 2001.